

SYSTEM ARCHITECTURE DOCUMENT (SAD)

EV Charging Station Reservation System

PES University - Department of CSE

Student: Shriya S R

SRN: PES1UG24CS449

Problem Statement: #21 - Smart Cities, Transport & Logistics

High-level architecture and component responsibilities

1. Architecture Overview

The EV Charging Station Reservation System is organized as a layered web application. An EV Driver or Station Operator interacts through a web interface. Application services handle station discovery, connector compatibility, reservation consistency, pricing, and booking lifecycle rules. A persistent data layer stores users, stations, chargers, availability slots, and reservations.

2. Architectural Goals

Goal	Architectural implication
Correct reservations	Reservation creation must perform an atomic availability check so the same socket/slot cannot be booked twice
Compatibility	Connector matching remains a dedicated service rule rather than being duplicated across UI screens
Dynamic rates	Pricing logic is isolated so rate inputs/rules can evolve without rewriting reservation workflows
Security	Authentication and authorization are enforced at service boundaries, not only in the UI
Maintainability	Modules are separated by responsibility and communicate through clear service interfaces

3. High-Level Component Architecture

User / External Layer	Application Layer	Data Layer
EV Driver Search compatible stations Reserve slot View/cancel reservation	Web UI Driver and operator screens Authentication Service Login, session, access control Station & Availability Service Search stations, chargers, slots Compatibility Service CCS / CHAdeMO / Type 2 matching Reservation Service Conflict checks, booking lifecycle Pricing Service Dynamic power-rate calculation	Relational/Persistent Store Users Stations Chargers Availability slots Reservations Pricing rules
Station Operator Manage station information Manage charger availability Review reservations	Notification / Status Adapter Booking confirmation, cancellation, release/penalty status	Audit / Logs Authentication events Reservation events Errors and test evidence

Data flow for a reservation: the driver submits connector and desired slot -> Web UI sends the request to the Reservation Service -> Compatibility and Availability checks validate the charger -> Pricing Service calculates the applicable rate -> Reservation Service performs a conflict-safe write -> persistent store records the reservation -> UI/notification layer returns confirmation or a clear failure reason.

4. Component Responsibilities

Component	Responsibilities
Web UI	Collect inputs, display compatible chargers, show availability, present booking/pricing status, and provide operator views
Authentication Service	Authenticate users and enforce driver/operator permissions
Station & Availability Service	Maintain station/charger metadata and expose available slots
Compatibility Service	Match the vehicle connector type against supported charger types
Reservation Service	Create, cancel, complete, expire, and protect reservations from double booking
Pricing Service	Apply configured inputs/rules to calculate dynamic power rates
Persistent Store	Persist users, stations, chargers, slots, reservations, rates, and audit records

5. Key Architectural Decisions

- Reservation consistency is prioritized over optimistic UI state: the final availability check must occur when the reservation is committed.
- Connector compatibility is a separate rule so support for additional connector types can be added without redesigning the whole booking flow.
- Pricing is isolated from reservation storage so pricing rules can change independently of booking mechanics.
- Driver and Station Operator permissions are distinct to prevent unauthorized station/charger administration.
- Audit/log records support debugging, security review, and traceability.

6. Non-Functional Architecture Considerations

Performance: index station/charger/slot lookup fields and keep reservation transactions short.

Security: authenticate users, authorize role-specific operations, validate all input, and protect sensitive session data.

Reliability: preserve reservation state transitions and make failure responses explicit.

Scalability: application services can be scaled independently from persistent storage as usage grows.

Maintainability: keep business rules in services rather than embedding them directly in UI code.

7. Traceability to Lab Problem Statement

The architecture directly supports the problem statement's two stakeholder/actor groups (EV Driver and Station Operator), compatible charging sockets (CCS, CHAdeMO, Type 2), guaranteed time-slot reservation, and dynamic power-rate calculation. The test plan uses the same requirement themes for verification.